

Ejercicios Tema 1

1. Comprobador de Palíndromos

En este ejercicio vas a crear un pequeño programa que **determine si una palabra es un palíndromo**, es decir, si se lee igual de izquierda a derecha que de derecha a izquierda (ejemplo: “*radar*”, “*oso*”, “*reconocer*”).

1. **Pide una palabra** al usuario.
2. Convierte la palabra a minúsculas para evitar errores con mayúsculas.
3. **Compara la palabra original con su versión invertida** y muestra true o false dependiendo de si es o no un palíndromo.

Pista:

- `"hola"[::-1]` devuelve `"aloh"`.
- Esa notación (`[::-1]`) es una **extensión del slicing** que ya hemos visto (`[ : ]`):

Slicing [:]	Extrae una parte	"Python"[0:3]	Pyt
---------------	------------------	---------------	-----

- El primer “.” indica que se toma toda la cadena.
- El segundo “.” indica el paso.
- Si el paso es `-1`, recorre la cadena **en sentido inverso**.

Además...

Si quieres hacer más divertidas las salidas, puedes usar estos emojis:

🧩 Tabla de emojis y códigos Unicode

Emoji	Significado	Código Unicode para usar en Python	Ejemplo de uso
🧩	Resultado / ejercicio / lógica	<code>\U0001F9E9</code>	<code>print("\U0001F9E9 Resultado:", True)</code>
⚙️	Proceso / comprobación / condición	<code>\u2699</code>	<code>print("\u2699 Comprobando...")</code>
💡	Idea / pista / resultado calculado	<code>\U0001F4A1</code>	<code>print("\U0001F4A1 Resultado:", valor)</code>
🔍	Revisión / comprobación	<code>\U0001F50D</code>	<code>print("\U0001F50D Analizando:", valor)</code>
🧠	Evaluación lógica / razonamiento	<code>\U0001F9E0</code>	<code>print("\U0001F9E0 Resultado lógico:", True)</code>
⚖️	Igualdad / equivalencia / comparación	<code>\U0001F7F0</code>	<code>print("\U0001F7F0 Resultado:", True)</code>
✅	Correcto / verdadero	<code>\u2705</code>	<code>print("\u2705 True")</code>
❌	Incorrecto / falso	<code>\u274C</code>	<code>print("\u274C False")</code>

2. Contraseña con vocales y números

Crea un programa que pida una **contraseña** y compruebe si cumple **tres condiciones básicas**:

1. Tiene una **longitud de al menos 8 caracteres**.
2. Contiene **al menos una vocal** (a, e, i, o, u, en minúscula o mayúscula).
3. Contiene **al menos un número** (0–9).

El programa debe mostrar si cada condición se cumple y, al final, indicar si la contraseña es **segura** o **débil**.

Pista: puedes usar el operador `in` para comprobar si una letra o número está dentro de la cadena.

3. Casting (cambiar el tipo de dato de una variable a otro tipo compatible) y operadores con texto y números

En este ejercicio vas a practicar cómo Python trata de forma diferente los **números** y las **cadenas de texto**.

1. Pide al usuario un número **escrito como texto**, por ejemplo `"25"`.
2. Pide otro número **entero real**, por ejemplo `5`.
3. Convierte el primer valor (el texto) a número con `int()`, para poder hacer operaciones matemáticas.
4. Muestra en un **solo `print()`** dos resultados:
 - ♦ La **suma numérica** de ambos valores.
 - ♦ La **repetición del texto**, multiplicando la cadena por el número.

Pista:

- `"25"` es texto → no se puede sumar directamente con un número.
- `int("25")` convierte ese texto en un número entero.
- El operador `*` permite repetir cadenas (`"hola" * 3` → `"holaholahola"`).

4. Lógica booleana en una sola expresión

En este ejercicio vas a comprobar si un número cumple **dos condiciones al mismo tiempo**, usando operadores lógicos.

1. Pide al usuario un número entero.
2. Comprueba si **está entre 1 y 100** (ambos incluidos).
3. Comprueba además si **es múltiplo de 3 o de 5**.
4. Muestra el resultado de esa comprobación en pantalla, 3 veces, usando imprimir normal con variables, imprimir con f-string simple e imprimir con f-string con operación.

Pista:

- Para comprobar si un número está entre dos valores puedes usar una comparación encadenada:
`1 <= x <= 100`
- Para saber si un número es múltiplo de otro usamos el operador módulo `%`.
Ejemplo: `x % 3 == 0` significa "x es divisible por 3".

5. Casting con valores booleanos

Crea un programa que pida al usuario tres valores diferentes:

1. Un **texto**
2. Un **número entero**

3. Un número decimal

Después convierte cada uno a **tipo booleano** (`bool()`) y muestra si su valor es **Verdadero** (`True`) o **Falso** (`False`).

Pistas:

- En Python, **todo valor tiene una “equivalencia booleana”**.
- Se considera `False` cuando el valor está “vacío” o es **cero**, y `True` en los demás casos.

6. Comprobación de campos vacíos con booleanos

Crea un programa que pida tres datos al usuario:

1. Su **nombre**
2. Su **correo electrónico**
3. Su **edad**

Luego, convierte cada dato a **booleano** (`bool()`) y muestra si cada campo se ha rellenado o se ha dejado vacío.

Finalmente, muestra si **todos los campos están completos** (`True`) o **alguno falta** (`False`).

Pistas:

- `input()` devuelve una cadena vacía (`" "`) si el usuario pulsa Enter sin escribir nada.
- `bool(" ")` devuelve `False`, mientras que cualquier texto devuelve `True`.
- Puedes combinar condiciones con `and` para comprobar si todos los campos son verdaderos.

7. Comprobación de números positivos con booleanos

Crea un programa que pida **tres números enteros** y determine si **al menos dos de ellos son positivos**.

Para ello:

1. Convierte cada número a **booleano** (`bool()`) para saber si es **verdadero (positivo o distinto de 0)** o **falso (cero o negativo)**.
2. Muestra el valor booleano de cada número.
3. Muestra si **al menos dos** de ellos son positivos usando operadores lógicos (`and`, `or`).

 *Pistas:*

- En Python, `bool(0)` es `False` y `bool(cualquier otro número)` es `True`.
- Puedes usar expresiones combinadas como:
`(bool(a) and bool(b)) or (bool(a) and bool(c)) or (bool(b) and bool(c))`
para saber si **dos o más son verdaderos**.

8. Comparación de edades

Crea un programa que pida las edades de **dos personas** y muestre, con valores `True` o `False`, si:

1. La primera persona es **mayor** que la segunda.
2. La primera persona es **menor** que la segunda.
3. Ambas personas tienen **la misma edad**.

9. Cálculo del IMC (sin `if`)

Crea un programa que calcule el **Índice de Masa Corporal (IMC)** a partir del **peso (kg)** y la **altura (m)** de una persona.

1. Pide el peso y la altura al usuario.

2. Calcula el IMC con la fórmula:

$$IMC = \frac{peso}{altura^2}$$

3. Muestra el valor del IMC con **dos decimales**.

4. Imprime tres expresiones booleanas (**True** o **False**) indicando si el IMC es:

- **Bajo peso** → menor que 18.5
- **Peso normal** → entre 18.5 y 24.9
- **Sobrepeso** → mayor o igual a 25

Pistas:

- Usa el operador de potencia ****** para elevar la altura al cuadrado.
- Usa comparaciones encadenadas como **18.5 <= imc <= 24.9**.

10.Ejercicio: Mostrar una tabla con nombres y edades usando caracteres de escape

Crea un programa en Python que muestre una pequeña tabla con los nombres y edades de dos personas.

El texto debe aparecer alineado en columnas, utilizando los caracteres especiales de escape:

- **\t** → para insertar tabulaciones (espacios amplios entre columnas).
- **\n** → para insertar saltos de línea.

11. Combinación alterna de letras (cadenas de 5 caracteres)

Crea un programa que trabaje con dos textos de exactamente 5 caracteres cada uno (*recuerda: el espacio también cuenta como un carácter*).

El programa debe hacer lo siguiente:

1. Imprimir las dos cadenas separadas por una barra vertical (|).
2. Crear una nueva cadena combinada tomando letras alternas:
 - 1ª letra de la primera cadena
 - 2ª letra de la segunda cadena
 - 3ª letra de la primera cadena
 - 4ª letra de la segunda cadena
3. Mostrar la nueva palabra en mayúsculas.
4. Imprimir todas las cadenas (las dos originales y la nueva) separadas por puntos (.).

Pistas:

- Usa indexación (`texto[0]`, `texto[1]`, ...) y concatenación (+).
- No necesitas `if` ni bucles.
- Si el texto tiene menos o más de 5 caracteres, el resultado no será correcto.

Concepto	Ejemplo
Indexación	<code>texto[0]</code> , <code>texto[1]</code>
Concatenación	<code>texto1[0] + texto2[1]</code>
<code>.upper()</code>	<code>nueva_cadena.upper()</code>
<code>sep</code> en <code>print()</code>	<code>`sep="`</code>
Control de longitud	trabajar con cadenas de tamaño fijo (5 caracteres)

12. Combinación e inversión de cadenas

Partiendo de dos cadenas de texto introducidas por el usuario, crea un programa que:

1. **Imprima las cadenas separadas por un guion bajo (_).**
2. Tome las **3 primeras letras** de la primera palabra.
3. Tome las **3 últimas letras** de la segunda palabra.
4. Una ambas partes para formar una **nueva cadena**.
5. Muestre la **cadena resultante invertida** (al revés).
6. Finalmente, muestra **todas las cadenas (las originales y la nueva)** separadas por una coma.

Pistas:

- Usa **slicing** para obtener partes de las cadenas:
 - `texto[:3]` → primeras 3 letras
 - `texto[-3:]` → últimas 3 letras
- Para invertir una cadena, usa slicing con paso negativo:
 - `texto[::-1]`
- Usa el parámetro `sep` de `print()` para separar por guiones o comas.

Concepto	Ejemplo usado
<code>sep</code> con <code>print()</code>	<code>print(texto1, texto2, sep="_")</code>
Slicing por posición	<code>texto[:3]</code> , <code>texto[-3:]</code>
Slicing inverso	<code>texto[::-1]</code>
Concatenación	<code>parte1 + parte2</code>
f-strings	<code>f"Nueva cadena: {nueva_cadena}"</code>

13. Ejercicio: Manipulación de dos cadenas

Crea un programa que pida **dos cadenas de texto** al usuario y realice diferentes operaciones con ellas.

1. Muestra cuántas letras tiene cada una.
2. Crea una nueva cadena que esté formada por:
 - desde la **3ª letra** de la primera palabra hasta el final,
 - seguida de la **primera mitad** de la segunda palabra.
3. Muestra la nueva cadena resultante en mayúsculas.
4. Finalmente, imprime todas las cadenas separadas por un guion (-) en una sola línea.

Pistas:

- Recuerda que las posiciones en Python comienzan en **0**.
- Para obtener una parte de una cadena puedes usar slicing: `texto[inicio:fin]`.
- Para la mitad de una palabra puedes usar `len(texto)//2`.